

Course Name

**Design and Analysis
of Algorithm**

Submitted To:

**Dr/ Gamal Ebrahim
Eng/ Sally Shaker**

Submitted By:

**Mohamed Ahmed Abdelhamid
(22p0287)**

Task 4

Table of Contents

Table of Contents	i
1.0 Abstract	1
2.0 Introduction	2
3.0 Problem Description	2
3.1 Assumptions.....	2
4.0 How To Think About The Problem	3
5.0 Program Code	4
5.1 Pseudocode	4
5.3 Pseudocode Simplified Version	4
6.0 Complexity Analysis	5
6.1 Time Complexity:	5
6.2 Space Complexity:.....	5
7.0 Comparison With Another Algorithm	5
7.1 A* Algorithm	5
7.2 Breadth-First-Search (BFS)	6
7.3 Comparison Table.....	6
8.0 Conclusion.....	7
9.0 References.....	7

1.0 Abstract

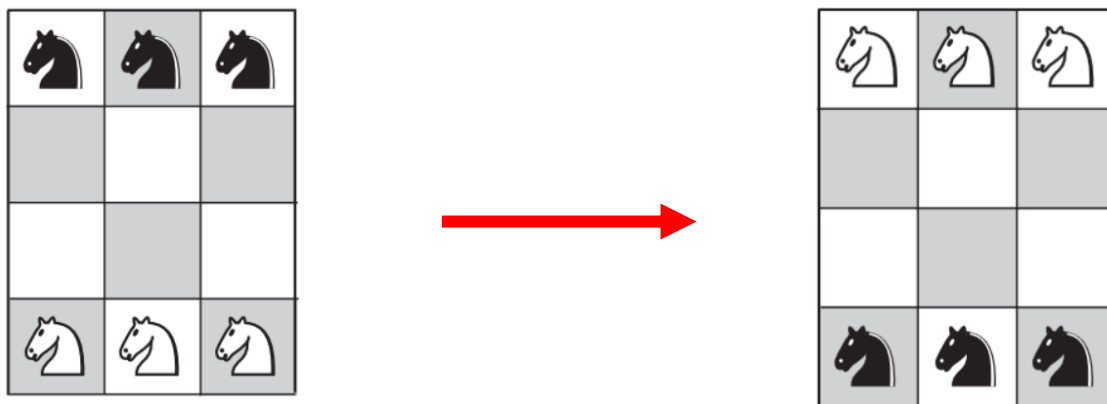
This report presents a solution to the Knight Exchange Problem using the A* search algorithm. The goal is to swap white and black knights on a limited 4x3 chessboard through valid knight moves requiring the least number of moves. We suggest a heuristic search strategy using path cost and a heuristic approximation of remaining misplacement to direct the search. It describes the problem formulation, algorithm design, pseudo-code, time complexity analysis, illustration of sample outputs and comparison of the program to another algorithm like brute-force BFS.

Design and Analysis of Algorithms

Task 4

2.0 Introduction

Search algorithms are the heart of solving pathfinding, and optimization. Here, we face a demanding problem as knights locate the chessboard trying to exchange positions by law. The restriction of knight movement generates a non-trivial solution space, prompting the employ of heuristic search methods. I selected the A* algorithm due to its efficiency and optimality when a suitable heuristic is applied.



3.0 Problem Description

There are six knights on a 3×4 chessboard: the three white knights are at the bottom row, and the three black knights are at the top row. Design an iterative improvement algorithm to exchange the knights to get the position shown on the right of the figure in the minimum number of knights moves, not allowing more than one knight on a square at any time.

3.1 Assumptions

- The initial positions are: Black pieces on top row (0th row) and the white knights in the bottom row (3rd row).
- Empty cells are represented by '.' (dots).
- Knights move according to chess rules (L-shaped: 8 possibilities).

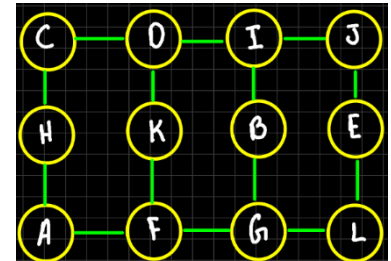
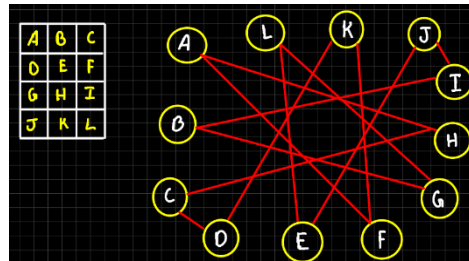
Design and Analysis of Algorithms

Task 4

- Only valid knight moves are allowed; no diagonal, horizontal or vertical direct movements.
- Every move is basically swapping a knight with an empty square such that movement is legal.

4.0 How To Think About The Problem

Step 1: Start with a feasible solution. Which means just reach a valid solution , not the best , that just follows the rules.



Step 2: Know each square of the grid is connected to which square. Look at the picture below to understand

For example, square “A” can go to square “F” or square “H”. We can simplify this network into the connection shown below.

This picture clearly shows the connection between the squares. From the picture, we can understand that squares “H” is reachable from square “C” or “A” and so on.

Now the first step in our code, we need to make a 3x4 grid and specify the connection of each square.

Second thing we have to do in the code, we must clarify the goal of each square. This can easily visualized in the picture shown in the previous page.

- Goal of White: A B C
- Goal of Black: J K L

Design and Analysis of Algorithms

Task 4

5.0 Program Code

5.1 Pseudocode

Initialize start board with B knights on top, W knights on bottom.

Define knight movement offsets (dx, dy).

Use priority queue to implement A*:

Nodes ordered by $f(n) = g(n) + h(n)$

Heuristic (h): Count of knights not in their goal row.

Loop:

Pop board with lowest cost from priority queue.

If goal is reached, update best result.

For each knight, generate all legal knight moves.

For each valid move:

Create new board with updated state.

Push into priority queue if not visited.

Print best move sequence and final board.

5.3 Pseudocode Simplified Version

- Initialize the board with the start configuration.
- Define a **heuristic**: the number of knights not in their target row.
- Use a **priority queue** to explore the board with the lowest cost first.
- For each board, check all knight positions and attempt valid knight moves.
- If a goal state is reached with fewer steps than the previously found solutions, update the best result.
- At the end, display the sequence of moves to reach the goal.

Design and Analysis of Algorithms

Task 4

6.0 Complexity Analysis

6.1 Time Complexity:

Worst-case nodes to explore: $O(b^d)$, where:

b is the branching factor (≤ 8 , max instalment one moves).

d is the depth of the shortest path.

Each board state can be encoded and the number of unique states is finite although it is large.

NOTE → A* improves over brute-force by pruning non-promising paths using the heuristic.

6.2 Space Complexity:

Priority queue size and visited set dominate: $O(N)$, where **N** is the number of unique board states.

Each board memory is 2D array of fixed size, memory per state can be low.

7.0 Comparison With Another Algorithm

There are different search methods for the knight exchange problem, the problem of finding the knight exchange. While this report focuses on the A* algorithm due to its optimality and admissibility properties, it's important to evaluate how it compares with alternative approaches, particularly Breadth-First Search (BFS), which is also commonly used for grid-based pathfinding.

7.1 A* Algorithm

A* is an informed search algorithm that combines the strengths of uniform-cost search and greedy best-first search. It uses a cost function $f(n) = g(n) + h(n)$ where:

The actual cost to get node **n** starting from the beginning point is denoted by **$g(n)$** .

The estimated cost to reach the goal state (heuristic) from node **n** is denoted by **$h(n)$** .

The heuristic measures incorrect row placements of knights where **n** indicates nodes in the search. It maintains admissibility while securing optimality.

Design and Analysis of Algorithms

Task 4

Because of its heuristic guidance, A* can significantly reduce the search space compared to blind search methods. Its design makes the algorithm ideal to solve problems with numerous possible moves and multiple optimal solution paths.

7.2 Breadth-First-Search (BFS)

Breadth-First Search is an uninformed search technique that searches all nodes at the current depth level before exploring nodes at the next depth level. BFS is promised to discover the shortest path in the number of move but fails to observe the cost or the structure of the problem space. Because of this it can be very in-efficient for larger state spaces as it visits many useless nodes before hitting the goal. In the context of the knight exchange problem, BFS would be to travel all possible board configurations level by level. This causes a high branching factor, particularly due to the complexity of a knight's movement and the number of permutations that exist on a 4x3 board. Thus, while correct, BFS is less efficient in time and space compared to A*.

7.3 Comparison Table

Feature	A* Search	Breadth-First Search (BFS)
Search Type	Informed	Uninformed
Uses Heuristic	Yes	No
Optimality	Yes (with admissible heuristic)	Yes (in uniform step-cost environments)
Completeness	Yes	Yes
Time Efficiency	Higher (guided by heuristic)	Lower (explores blindly)

Space Complexity	High (due to priority queue and visited states)	Very High (due to full tree expansion)
Suitability for Current Task	Excellent	Acceptable, but inefficient

8.0 Conclusion

This project successfully applied the A* search algorithm to solve a constrained pathfinding puzzle involving knight movements on a 3x4 chessboard. With an admissible heuristic (falsely row count) the solution was both a good and speedy one. Compared to uninformed methods, A* showed a significant reduction in computation and time. This outcome shows the ability of informed search to solve discrete puzzles and AI planning problems.

9.0 References

- <https://www.unipuzzle.com/chess-puzzles/chess.php?name=six-knights-guarinis-problem> (to simulate the problem and try)
- https://www.youtube.com/watch?v=EuDnVV8j_Zs (helpful tips on how to think about the problem)
- <https://www.geeksforgeeks.org/a-search-algorithm/>
- [Guarini's Puzzle, A Less Elegant Solution | Mostafa Touny](#)
- <https://puzzling.stackexchange.com/questions/25287/switch-the-knights/25301#25301>